

MLflow

Version: 3.11.1 (latest)

■ LLMs & Agents

Explore tools for LLM and agent observability, prompt management, foundation model deployment, and evaluation frameworks. Learn how to track, evaluate, and optimize your LLM applications and agent workflows with MLflow.

[Open Source](#) →

[MLflow on Databricks](#) →

■ Machine Learning

Access comprehensive guides for experiment tracking, model packaging, registry management, and deployment. Get started with MLflow's core functionality for traditional machine learning workflows, hyperparameter tuning, and model lifecycle management.

[Open Source](#) →

[MLflow on Databricks](#) →

MLflow

- MLflow is the largest open source **AI engineering platform for agents, LLMs, and ML models**.
- MLflow's tools for traditional machine learning and deep learning: **ML experiment tracking, model versioning, model deployment, and model evaluation**.
- **Core Feature:**
 - How to **log parameters, metrics, and a model** using the MLflow logging API
 - How to **navigate to a model** in the **MLflow UI**
 - How to **load a logged model** for inference
- **Documentation:** <https://mlflow.org/docs/latest/ml/>

Step 1 - Set up MLflow

- **> pip install mlflow**

```
import mlflow

mlflow.set_experiment("MLflow Quickstart")
```

Step 2 - Prepare training data

```
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load the Iris dataset
X, y = datasets.load_iris(return_X_y=True)

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define the model hyperparameters
params = {
    "solver": "lbfgs",
    "max_iter": 1000,
    "random_state": 8888,
}
```

Step 3 - Train a model with MLflow Autologging

- In this step, we train the model and **log the model** and **its metadata** to MLflow.
- The easiest way to do this is to using **MLflow's Autologging feature**.

```
# Enable autologging for scikit-Learn  
mlflow.sklearn.autolog()
```

```
# Just train the model normally  
lr = LogisticRegression(**params)  
lr.fit(X_train, y_train)
```

- Now, you can **focus on training the model**, and **MLflow will take care of the rest**:
 - Saving the trained model.
 - Recording the model's performance metrics during training, such as accuracy, precision, AUC curve.
 - Logging hyperparameter values used to train the model.
 - Track metadata such as input data format, user, timestamp, etc.

Automatic Logging with MLflow Tracking

- Auto logging is a powerful feature that allows you to **log metrics, parameters, and models** **without the need for explicit log statements**. All you need to do is to call **mlflow.autolog()** **before** your training code.

```
import mlflow

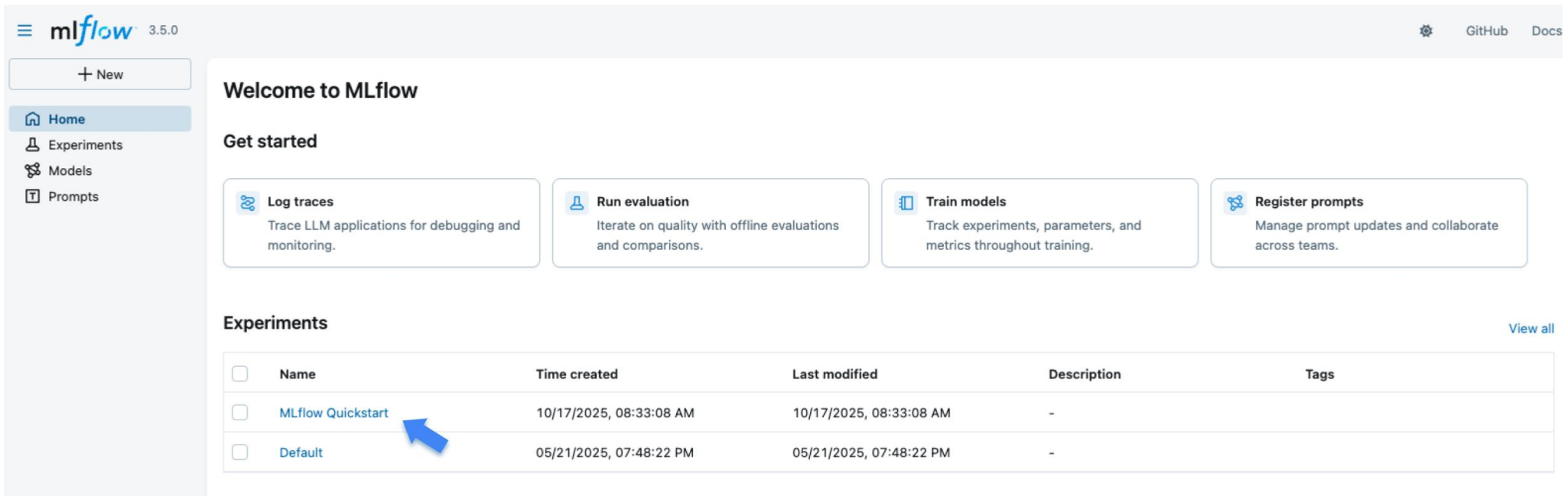
mlflow.autolog()

with mlflow.start_run():
    # your training code goes here
    ...
```

- **Metrics** - MLflow pre-selects a set of metrics to log, based on what model and library you use
- **Parameters** - hyper params specified for the training, plus default values provided by the library if not explicitly set
- **Model Signature** - logs **Model signature** instance, which describes input and output schema of the model
- **Artifacts** - e.g. model checkpoints
- **Dataset** - dataset object used for training (if applicable), such as *tensorflow.data.Dataset*

Step 4 - View the Run in the MLflow UI

- In Terminal: `> mlflow ui` OR `mlflow server --port 5000`
- Then, go to: <http://127.0.0.1:5000/>
- Open a new terminal, and run the code file.
- When opening the site, you will see a screen similar to the following:
- The "Experiments" section shows a list of (recently created) experiments. Click on the **"MLflow Quickstart" experiment**.



The screenshot displays the MLflow 3.5.0 user interface. On the left is a sidebar with navigation links: Home (selected), Experiments, Models, and Prompts. The main content area is titled 'Welcome to MLflow' and includes a 'Get started' section with four cards: 'Log traces', 'Run evaluation', 'Train models', and 'Register prompts'. Below this is the 'Experiments' section, which contains a table of experiments. A blue arrow points to the 'MLflow Quickstart' experiment in the table.

☐	Name	Time created	Last modified	Description	Tags
☐	MLflow Quickstart	10/17/2025, 08:33:08 AM	10/17/2025, 08:33:08 AM	-	
☐	Default	05/21/2025, 07:48:22 PM	05/21/2025, 07:48:22 PM	-	

Step 4 - View the Run in the MLflow UI

- The training Run created by MLflow is listed in the table. **Click the run to view the details.**

The screenshot shows the MLflow UI interface. On the left is a sidebar with navigation links: Home, Experiments (selected), Models, and Prompts. The main area displays the 'MLflow Quickstart' experiment under the 'Machine learning' category. It includes a search bar with the query 'metrics.rmse < 1 and params.model = "tree"', filters for 'Time created', 'State: Active', and 'Datasets', and a 'Sort: Created' dropdown. Below these are controls for 'Columns', 'Expand rows', and 'Group by'. A table lists the runs, with the first row highlighted in blue and a blue arrow pointing to its name, 'thundering-fly-695'.

☐	Run Name	Created	Dataset	Duration	Source	Models
☐	thundering-fly-695	2 hours ago	dataset (aa7a0237) Eval , dataset (f...	2.0s	ipykern...	model

Step 4 - View the Run in the MLflow UI

- The Run detail page shows an overview of the run, **its recorded metrics, hyper-parameters, tags, and more**. Play around with the UI to see the different views and features.

The screenshot displays the MLflow UI interface for a specific run. The left sidebar contains navigation links for Home, Experiments, Models, and Prompts. The main content area is titled 'thundering-fly-695' and includes tabs for Overview, Model metrics, System metrics, Traces, and Artifacts. The Overview tab is active, showing a description field (currently empty), a table of 7 metrics, and a table of 15 parameters. The right sidebar provides additional details about the run, including its creation time, creator, experiment ID, status (Finished), run ID, duration, source, registered prompts, datasets, tags, and registered models.

MLflow Quickstart > Runs > thundering-fly-695

Overview | Model metrics | System metrics | Traces | Artifacts

Description [✎](#)
No description

Metrics (7)

Metric	Value	Models
training_score	0.975	model
training_roc_auc	0.9981238273921201	model
training_accuracy_score	0.975	model
training_precision_score	0.9767857142857144	model
training_log_loss	0.13562857756813249	model
training_f1_score	0.9749882794186592	model
training_recall_score	0.975	model

Parameters (15)

Parameter	Value
tol	0.0001
penalty	l2
class_weight	None
random_state	8888

About this run

Created at: 10/17/2025, 08:33:11 AM
Created by: [admin](#)
Experiment ID: [379351188730453731](#) [🔗](#)
Status: ✔ Finished
Run ID: [43dc5b01bc994b198503bb0b1eed34d](#) [🔗](#)
Duration: 2.0s
Source: [📄 ipykernel_launcher.py](#)
Registered prompts: —

Datasets
[📄 dataset \(aa7a0237\)](#) Eval [+1](#)

Tags
`estimator_class: sklearn.linear_model._logistic...`
`estimator_name: LogisticRegression` [✎](#)

Registered models
None

Step 4 - View the Run in the MLflow UI

- Scroll down to the "Model" section and you should see the model that was logged during training. Click on the model to view the details.

Logged models (1)



Model attributes

Type	Step	Model name	Status	Created	Registered models
Output	0	 model	 Ready	2 hours ago	-



Step 4 - View the Run in the MLflow UI

- The **model page** displays similar metadata such as **performance metrics and hyper-parameters**.
- It also includes an **"Artifacts" section** that lists the **files that were logged during training**. You can also **see environment information such as the Python version and dependencies**, which are stored for reproducibility.

MLflow Quickstart > Models >

 **model**

Overview Traces **Artifacts**

 MLmodel
 conda.yaml
 model.pkl
 python_env.yaml
 requirements.txt

MLmodel 985B

Path: file:///Users/yuki.watanabe/Workspace/mlflow-playground/mlflow-3/mlruns/3793

artifact_path: file:///Users/yuki.watanabe/Workspace/mlflow-playground

flavors:

python_function:

env:

conda: conda.yaml

virtualenv: python_env.yaml

loader_module: mlflow.sklearn

model_path: model.pkl

predict_fn: predict

python_version: 3.12.8

Step 5 - Log a model and metadata manually

- This is useful when you want to have **more control over the logging process**.
- **The steps that we will take are:**
 - Initiate an MLflow **run** context to **start a new run that we will log the model and metadata to**.
 - **Train** and **test** the model.
 - **Log** model **parameters** and performance **metrics**.
 - **Tag** the run **for easy retrieval**.

Step 5 - Log a model and metadata manually

```
# Start an MLflow run
with mlflow.start_run():
    # Log the hyperparameters
    mlflow.log_params(params)

    # Train the model
    lr = LogisticRegression(**params)
    lr.fit(X_train, y_train)

    # Log the model
    model_info = mlflow.sklearn.log_model(sk_model=lr, name="iris_model")

    # Predict on the test set, compute and log the loss metric
    y_pred = lr.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    mlflow.log_metric("accuracy", accuracy)

    # Optional: Set a tag that we can use to remind ourselves what this run was for
    mlflow.set_tag("Training Info", "Basic LR model for iris data")
```

Step 6 - Load the model back for inference

- After logging the model, we can **perform inference** by:
 - Loading the model using MLflow's **pyfunc** flavor.
 - Running **Predict** on new data using the loaded model.
- **To load the model as a native scikit-learn model**, use **`mlflow.sklearn.load_model(model_info.model_uri)`** instead of the **pyfunc** flavor.

```
# Load the model back for predictions as a generic Python Function model
loaded_model = mlflow.pyfunc.load_model(model_info.model_uri)

predictions = loaded_model.predict(X_test)

iris_feature_names = datasets.load_iris().feature_names

result = pd.DataFrame(X_test, columns=iris_feature_names)
result["actual_class"] = y_test
result["predicted_class"] = predictions

result[:4]
```

The output of this code will look something like this:

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	actual_class	predicted_class
6.1	2.8	4.7	1.2	1	1
5.7	3.8	1.7	0.3	0	0
7.7	2.6	6.9	2.3	2	2
6.0	2.9	4.5	1.5	1	1

Thank you

- **Documentation:** <https://mlflow.org/docs/latest/ml/>
- Quick Start: <https://mlflow.org/docs/latest/ml/getting-started/quickstart/>
- Autologging: <https://mlflow.org/docs/latest/ml/tracking/autolog/>